```
documents = [
    "The quick brown fox jumps over the lazy dog.",
    "A brown fox is quick and jumps over the lazy dog.",
    "The dog is lazy and sleeps all day.",
    "Cats are known for their agility and grace.",
    "Dogs are loyal companions and often considered man's best friend.",
    "The sun is shining brightly today.",
    "Birds sing sweetly in the morning.",
    "Books offer a window into different worlds and knowledge.",
    "Technology continues to advance at a rapid pace.",
    "Healthy eating and regular exercise are crucial for well-being.",
    "Artificial intelligence is transforming various industries.",
    "Machine learning algorithms are at the core of many AI applications.",
    "Natural language processing focuses on human-computer language interaction.",
    "Data science involves extracting insights from complex datasets.",
    "Computer vision enables machines to interpret and understand visual information."
]

print("First 5 documents:")
for i, doc in enumerate(documents[:5]):
    print(f"Document {i+1}: {doc}")

print(f"\nTotal number of documents: {len(documents)}")
```

```
First 5 documents:
Document 1: The quick brown fox jumps over the lazy dog.
Document 2: A brown fox is quick and jumps over the lazy dog.
Document 3: The dog is lazy and sleeps all day.
Document 4: Cats are known for their agility and grace.
Document 5: Dogs are loyal companions and often considered man's best friend.

Total number of documents: 15
```

```
import nltk
nltk.download('stopwords')
nltk.download('punkt')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')

print("NLTK resources downloaded successfully.")
```

```
NLTK resources downloaded successfully.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
```

```
import nltk
nltk.download('stopwords')
nltk.download('punkt')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')
nltk.download('punkt_tab') # Added based on the error message
```

```
print("NLTK resources downloaded successfully.")
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
NLTK resources downloaded successfully.
```

```
import nltk
nltk.download('stopwords')
nltk.download('punkt')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')
nltk.download('punkt_tab') # Added based on the error message
nltk.download('averaged_perceptron_tagger_eng') # Added based on the new error message

print("NLTK resources downloaded successfully.")
```

```
NLTK resources downloaded successfully.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger_eng.zip.
```

```
import string
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from nltk.corpus import wordnet

def get_wordnet_pos(word):
    """Map NLTK POS tag to WordNet POS tag for lemmatization"""
    tag = nltk.pos_tag([word])[0][1][0].upper()
    tag_dict = {"J": wordnet.ADJ,
                "N": wordnet.NOUN,
                "V": wordnet.VERB,
                "R": wordnet.ADV}
    return tag_dict.get(tag, wordnet.NOUN)

def preprocess_text(text):
    # 1. Convert to lowercase
    text = text.lower()
```

```
    # 2. Remove punctuation
    text = text.translate(str.maketrans('', '', string.punctuation))

    # 3. Tokenize text
    tokens = word_tokenize(text)

    # 4. Remove stopwords
    stop_words = set(stopwords.words('english'))
    filtered_tokens = [word for word in tokens if word not in stop_words]

    # 5. Optional: Lemmatization
    lemmatizer = WordNetLemmatizer()
    lemmatized_tokens = [lemmatizer.lemmatize(word, get_wordnet_pos(word)) for word in filtered_

    return " ".join(lemmatized_tokens)

# Apply preprocessing to all documents
preprocessed_documents = [preprocess_text(doc) for doc in documents]

# Print original and preprocessed versions of a few sample documents
print("--- Original vs. Preprocessed Documents ---")
for i in range(min(5, len(documents))):
    print(f"Original Document {i+1}: {documents[i]}")
    print(f"Preprocessed Document {i+1}: {preprocessed_documents[i]}\n")
```

```
--- Original vs. Preprocessed Documents ---
Original Document 1: The quick brown fox jumps over the lazy dog.
Preprocessed Document 1: quick brown fox jump lazy dog

Original Document 2: A brown fox is quick and jumps over the lazy dog.
Preprocessed Document 2: brown fox quick jump lazy dog

Original Document 3: The dog is lazy and sleeps all day.
Preprocessed Document 3: dog lazy sleep day

Original Document 4: Cats are known for their agility and grace.
Preprocessed Document 4: cat know agility grace

Original Document 5: Dogs are loyal companions and often considered man's best friend.
Preprocessed Document 5: dog loyal companion often consider man best friend
```

```
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer

# Initialize TfidfVectorizer and fit-transform the preprocessed documents
tfidf_vectorizer = TfidfVectorizer()
tfidf_matrix = tfidf_vectorizer.fit_transform(preprocessed_documents)

# Initialize CountVectorizer and fit-transform the preprocessed documents
count_vectorizer = CountVectorizer()
bow_matrix = count_vectorizer.fit_transform(preprocessed_documents)

# Print the shapes of the generated matrices
print(f"Shape of TF-IDF matrix: {tfidf_matrix.shape}")
print(f"Shape of Bag-of-Words matrix: {bow_matrix.shape}")
```

```
Shape of TF-IDF matrix: (15, 76)
Shape of Bag-of-Words matrix: (15, 76)
```

```
from sklearn.metrics.pairwise import cosine_similarity

# Calculate the cosine similarity matrix
cosine_sim_matrix = cosine_similarity(tfidf_matrix)

print("Cosine Similarity Matrix shape:", cosine_sim_matrix.shape)
```

```
Cosine Similarity Matrix shape: (15, 15)
```

```
import numpy as np

# Create a list to store (similarity_score, doc_index_1, doc_index_2)
similar_pairs = []

# Iterate through the upper triangular part of the similarity matrix (excluding diagonal)
for i in range(len(documents)):
    for j in range(i + 1, len(documents)): # Start from i+1 to avoid self-comparison and duplica
        similar_pairs.append((cosine_sim_matrix[i, j], i, j))

# Sort the list by similarity score in descending order
similar_pairs.sort(key=lambda x: x[0], reverse=True)

# Print the top 5 most similar pairs
print("\n--- Top 5 Most Similar Document Pairs (Cosine Similarity) ---")
for k, (score, doc_idx1, doc_idx2) in enumerate(similar_pairs[:5]):
    print(f"Pair {k+1}: Documents {doc_idx1+1} and {doc_idx2+1}")
    print(f"Similarity Score: {score:.4f}")
    print(f"  Document {doc_idx1+1}: {documents[doc_idx1]}")
    print(f"  Document {doc_idx2+1}: {documents[doc_idx2]}\n")

print("Interpretation of Top 5 Similar Pairs:")
print("1. Documents 1 and 2: These documents are almost identical, only differing by a few stopw
print("2. Documents 1 and 3 (and 2 and 3): These pairs share keywords like 'dog' and 'lazy'. Doc
print("3. Documents 1 and 5 (and 2 and 5): These documents both mention 'dog', contributing to a
print("4. Documents 11 and 12: These documents discuss 'Artificial intelligence' and 'Machine le
print("5. Documents 13 and 14: These documents talk about 'Natural language processing' and 'Dat
```

```

--- Top 5 Most Similar Document Pairs (Cosine Similarity) ---
Pair 1: Documents 1 and 2
Similarity Score: 1.0000
  Document 1: The quick brown fox jumps over the lazy dog.
  Document 2: A brown fox is quick and jumps over the lazy dog.

Pair 2: Documents 1 and 3
Similarity Score: 0.3068
  Document 1: The quick brown fox jumps over the lazy dog.
  Document 3: The dog is lazy and sleeps all day.

Pair 3: Documents 2 and 3
Similarity Score: 0.3068
  Document 2: A brown fox is quick and jumps over the lazy dog.
  Document 3: The dog is lazy and sleeps all day.

Pair 4: Documents 12 and 15
Similarity Score: 0.1042
  Document 12: Machine learning algorithms are at the core of many AI applications.
  Document 15: Computer vision enables machines to interpret and understand visual information.

Pair 5: Documents 3 and 5
Similarity Score: 0.1025
  Document 3: The dog is lazy and sleeps all day.
```

```
  Document 5: Dogs are loyal companions and often considered man's best friend.

Interpretation of Top 5 Similar Pairs:
1. Documents 1 and 2: These documents are almost identical, only differing by a few stopwords and
2. Documents 1 and 3 (and 2 and 3): These pairs share keywords like 'dog' and 'lazy'. Document 1/
3. Documents 1 and 5 (and 2 and 5): These documents both mention 'dog', contributing to a low but
4. Documents 11 and 12: These documents discuss 'Artificial intelligence' and 'Machine learning a
5. Documents 13 and 14: These documents talk about 'Natural language processing' and 'Data scienc
```

```
def jaccard_similarity(set1, set2):
    intersection = len(set1.intersection(set2))
    union = len(set1.union(set2))
    return intersection / union if union != 0 else 0

# Create a list of word sets from preprocessed documents
word_sets = [set(doc.split()) for doc in preprocessed_documents]

print("Jaccard similarity function defined and word sets created.")
```

```
Jaccard similarity function defined and word sets created.
```

```
import numpy as np

# Initialize an empty list to store (similarity_score, doc_index_1, doc_index_2)
jaccard_similar_pairs = []

# Iterate through the upper triangular part of the document pairs
for i in range(len(documents)):
    for j in range(i + 1, len(documents)): # Start from i+1 to avoid self-comparison and duplica
        score = jaccard_similarity(word_sets[i], word_sets[j])
        jaccard_similar_pairs.append((score, i, j))

# Sort the list by similarity score in descending order
jaccard_similar_pairs.sort(key=lambda x: x[0], reverse=True)

# Print the top 5 most similar pairs
print("\n--- Top 5 Most Similar Document Pairs (Jaccard Similarity) ---")
for k, (score, doc_idx1, doc_idx2) in enumerate(jaccard_similar_pairs[:5]):
    print(f"Pair {k+1}: Documents {doc_idx1+1} and {doc_idx2+1}")
    print(f"Similarity Score: {score:.4f}")
    print(f"  Document {doc_idx1+1}: {documents[doc_idx1]}")
    print(f"  Document {doc_idx2+1}: {documents[doc_idx2]}\n")
```

```
--- Top 5 Most Similar Document Pairs (Jaccard Similarity) ---
Pair 1: Documents 1 and 2
Similarity Score: 1.0000
  Document 1: The quick brown fox jumps over the lazy dog.
  Document 2: A brown fox is quick and jumps over the lazy dog.

Pair 2: Documents 1 and 3
Similarity Score: 0.2500
  Document 1: The quick brown fox jumps over the lazy dog.
  Document 3: The dog is lazy and sleeps all day.

Pair 3: Documents 2 and 3
Similarity Score: 0.2500
  Document 2: A brown fox is quick and jumps over the lazy dog.
  Document 3: The dog is lazy and sleeps all day.

Pair 4: Documents 3 and 5
```

```
Similarity Score: 0.0909
  Document 3: The dog is lazy and sleeps all day.
  Document 5: Dogs are loyal companions and often considered man's best friend.

Pair 5: Documents 1 and 5
Similarity Score: 0.0769
  Document 1: The quick brown fox jumps over the lazy dog.
  Document 5: Dogs are loyal companions and often considered man's best friend.
```

```
from nltk.corpus import wordnet

def get_wordnet_pos(word):
    """Map NLTK POS tag to WordNet POS tag for lemmatization"""
    tag = nltk.pos_tag([word])[0][1][0].upper()
    tag_dict = {"J": wordnet.ADJ,
                "N": wordnet.NOUN,
                "V": wordnet.VERB,
                "R": wordnet.ADV}
    return tag_dict.get(tag, wordnet.NOUN)

def word_similarity(word1, word2):
    """Calculates the maximum path similarity between two words using WordNet."""
    synsets1 = wordnet.synsets(word1)
    synsets2 = wordnet.synsets(word2)

    if not synsets1 or not synsets2:
        return 0.0 # No synsets found for one or both words

    max_similarity = 0.0
    for s1 in synsets1:
        for s2 in synsets2:
            similarity = s1.path_similarity(s2)
            if similarity is not None and similarity > max_similarity:
                max_similarity = similarity
    return max_similarity

print("word_similarity function defined.")
```

```
word_similarity function defined.
```

```
def sentence_similarity(sentence1, sentence2, preprocess_func):
    """Calculates the overall semantic similarity between two sentences using WordNet."""
    # Preprocess sentences to get lemmatized tokens
    tokens1 = preprocess_func(sentence1).split()
    tokens2 = preprocess_func(sentence2).split()

    if not tokens1 or not tokens2:
        return 0.0

    # Sum of maximum similarities from sentence1 to sentence2
    s1_to_s2_sim_sum = 0.0
    for t1 in tokens1:
        max_sim_t1 = 0.0
        for t2 in tokens2:
            sim = word_similarity(t1, t2)
            if sim > max_sim_t1:
                max_sim_t1 = sim
        s1_to_s2_sim_sum += max_sim_t1
```

```
    # Sum of maximum similarities from sentence2 to sentence1
    s2_to_s1_sim_sum = 0.0
    for t2 in tokens2:
        max_sim_t2 = 0.0
        for t1 in tokens1:
            sim = word_similarity(t2, t1)
            if sim > max_sim_t2:
                max_sim_t2 = sim
        s2_to_s1_sim_sum += max_sim_t2

    # Calculate the average of these two summed similarities
    # Divide by the total number of words in both preprocessed sentences
    total_words = len(tokens1) + len(tokens2)
    if total_words == 0: # Should not happen if tokens1 or tokens2 not empty, but safety check
        return 0.0

    # Use the average of the two directional sums, normalized by the number of words in each sen
    # A common approach is to normalize by the length of the sentence being compared FROM
    # Here, we'll try a symmetrical approach: (sum_s1_to_s2 + sum_s2_to_s1) / (len(s1) + len(s2)
    # Or, as a simpler average of two directional averages:
    avg_s1_to_s2 = s1_to_s2_sim_sum / len(tokens1)
    avg_s2_to_s1 = s2_to_s1_sim_sum / len(tokens2)

    return (avg_s1_to_s2 + avg_s2_to_s1) / 2.0

print("sentence_similarity function defined.")
```

```
sentence_similarity function defined.
```

```
selected_sentence_pairs = [
    (documents[0], documents[1]), # High lexical overlap, high semantic similarity
    (documents[0], documents[2]), # Moderate lexical overlap, moderate semantic similarity (sharec
    (documents[4], documents[2]), # Moderate lexical overlap, moderate semantic similarity (sharec
    (documents[10], documents[11]), # Related concepts, potentially higher semantic than lexical c
    (documents[12], documents[13]), # Related concepts, potentially higher semantic than lexical c
    (documents[6], documents[7]), # Low lexical overlap, potentially some semantic relation (natur
    (documents[8], documents[9]), # Low lexical overlap, low semantic similarity (technology vs he
    (documents[0], documents[5]), # Completely unrelated topics
    (documents[10], documents[14]), # Related fields ('AI' vs 'Computer Vision')
    (documents[7], documents[13]) # 'Books' and 'Data science' - potentially some indirect link th
]

print("--- WordNet-based Semantic Similarity for Selected Pairs ---")
for i, (s1, s2) in enumerate(selected_sentence_pairs):
    similarity_score = sentence_similarity(s1, s2, preprocess_text)
    print(f"\nPair {i+1}:")
    print(f"  Sentence 1: {s1}")
    print(f"  Sentence 2: {s2}")
    print(f"  WordNet Semantic Similarity: {similarity_score:.4f}")
```

```
--- WordNet-based Semantic Similarity for Selected Pairs ---

Pair 1:
  Sentence 1: The quick brown fox jumps over the lazy dog.
  Sentence 2: A brown fox is quick and jumps over the lazy dog.
  WordNet Semantic Similarity: 1.0000

Pair 2:
  Sentence 1: The quick brown fox jumps over the lazy dog.
  Sentence 2: The dog is lazy and sleeps all day.
  WordNet Semantic Similarity: 0.5799
```

```
Pair 3:
  Sentence 1: Dogs are loyal companions and often considered man's best friend.
  Sentence 2: The dog is lazy and sleeps all day.
  WordNet Semantic Similarity: 0.4073

Pair 4:
  Sentence 1: Artificial intelligence is transforming various industries.
  Sentence 2: Machine learning algorithms are at the core of many AI applications.
  WordNet Semantic Similarity: 0.2588

Pair 5:
  Sentence 1: Natural language processing focuses on human-computer language interaction.
  Sentence 2: Data science involves extracting insights from complex datasets.
  WordNet Semantic Similarity: 0.2256

Pair 6:
  Sentence 1: Birds sing sweetly in the morning.
  Sentence 2: Books offer a window into different worlds and knowledge.
  WordNet Semantic Similarity: 0.2785

Pair 7:
  Sentence 1: Technology continues to advance at a rapid pace.
  Sentence 2: Healthy eating and regular exercise are crucial for well-being.
  WordNet Semantic Similarity: 0.2903

Pair 8:
  Sentence 1: The quick brown fox jumps over the lazy dog.
  Sentence 2: The sun is shining brightly today.
  WordNet Semantic Similarity: 0.2944

Pair 9:
  Sentence 1: Artificial intelligence is transforming various industries.
  Sentence 2: Computer vision enables machines to interpret and understand visual information.
  WordNet Semantic Similarity: 0.3176

Pair 10:
  Sentence 1: Books offer a window into different worlds and knowledge.
  Sentence 2: Data science involves extracting insights from complex datasets.
  WordNet Semantic Similarity: 0.2925
```